

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)
```

```
iris = load_iris()

X = iris.data
y = iris.target

feature_names = iris.feature_names
target_names = iris.target_names

print("Dataset shape:", X.shape)
print("Features:", feature_names)
print("Classes:", target_names)
```

```
Dataset shape: (150, 4)
Features: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Classes: ['setosa' 'versicolor' 'virginica']
```

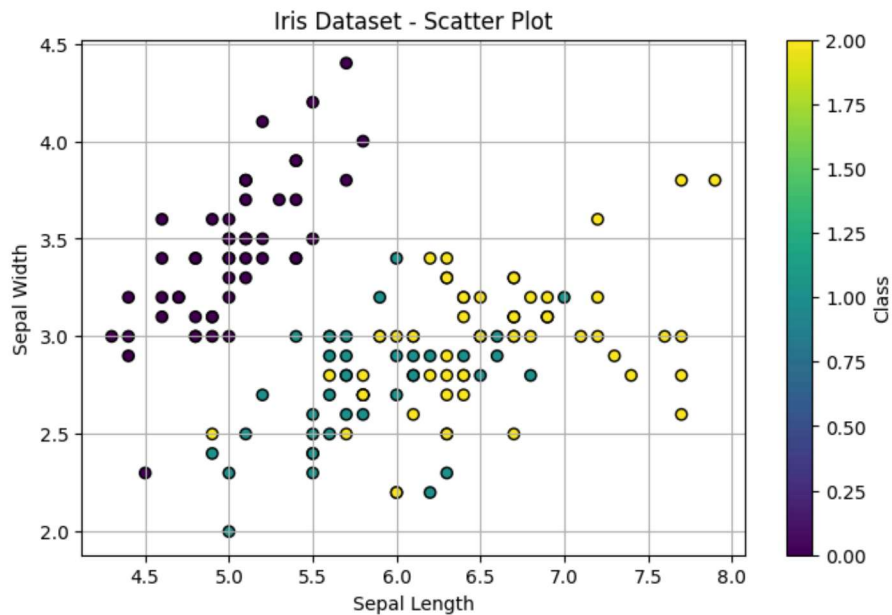
```
plt.figure(figsize=(8, 5))

plt.scatter(
    X[:, 0],
    X[:, 1],
    c=y,
    cmap='viridis',
    edgecolors='black'
)

plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.title("Iris Dataset - Scatter Plot")

plt.grid(True)
plt.colorbar(label="Class")

plt.show()
```



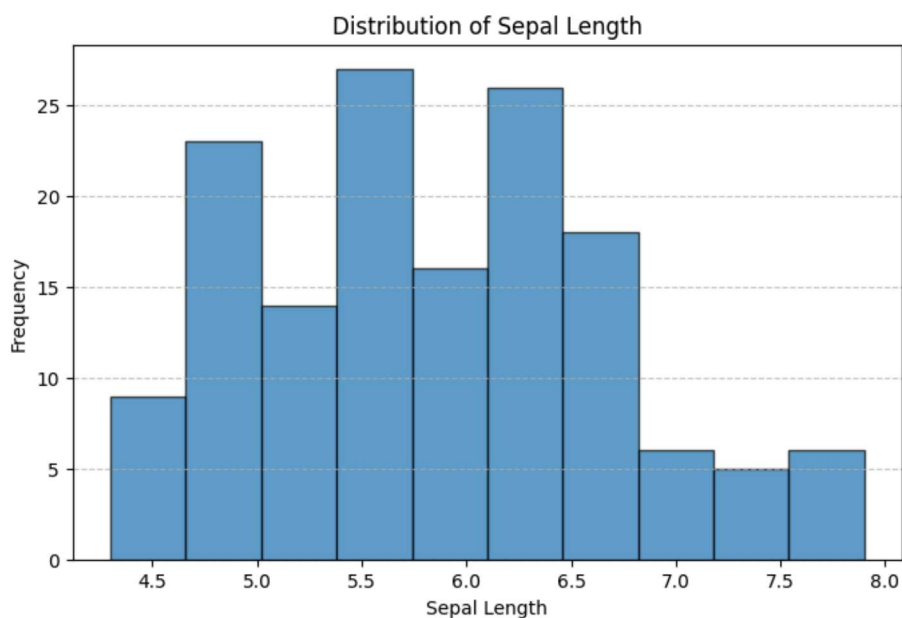
```
plt.figure(figsize=(8, 5))

plt.hist(
    X[:, 0],
    bins=10,
    edgecolor='black',
    alpha=0.7
)

plt.xlabel("Sepal Length")
plt.ylabel("Frequency")
plt.title("Distribution of Sepal Length")

plt.grid(axis='y', linestyle='--', alpha=0.7)

plt.show()
```



```
class_counts = np.bincount(y)

plt.figure(figsize=(7, 7))

plt.pie(
    class_counts,
    labels=target_names,
```

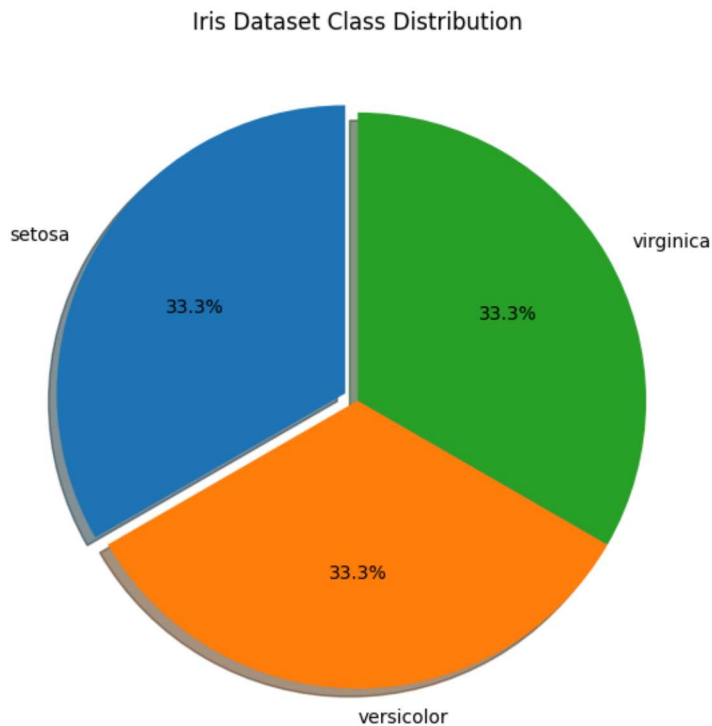
```

        autopct='%1.1f%%',
        startangle=90,
        explode=[0.05, 0, 0],
        shadow=True
    )

plt.title("Iris Dataset Class Distribution")

plt.show()

```



```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

```

```

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

```

```

Training samples: 120
Testing samples: 30

```

```

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

```

models = {
    "Logistic Regression": LogisticRegression(max_iter=200),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        random_state=42
    )
}

```

```

results = []

for name, model in models.items():

    if name in ["Logistic Regression", "KNN"]:

```

```

if name in ['Logistic Regression', 'KNN']:
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
else:
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

results.append([
    name,
    accuracy,
    precision,
    recall,
    f1
])

```

```

results_df = pd.DataFrame(
    results,
    columns=[
        "Model",
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score"
    ]
)

print(results_df)

```

	Model	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.933333	0.933333	0.933333	0.933333
1	Decision Tree	0.933333	0.933333	0.933333	0.933333
2	KNN	0.933333	0.944444	0.933333	0.932660
3	Random Forest	0.900000	0.902357	0.900000	0.899749

```

plt.figure(figsize=(10, 6))

bars = plt.bar(
    results_df["Model"],
    results_df["Accuracy"],
    edgecolor='black',
    alpha=0.8
)

plt.xlabel("Machine Learning Model")
plt.ylabel("Accuracy")
plt.title("Accuracy Comparison of Machine Learning Models")

plt.ylim(0, 1.1)
plt.grid(axis='y', linestyle='--', alpha=0.7)

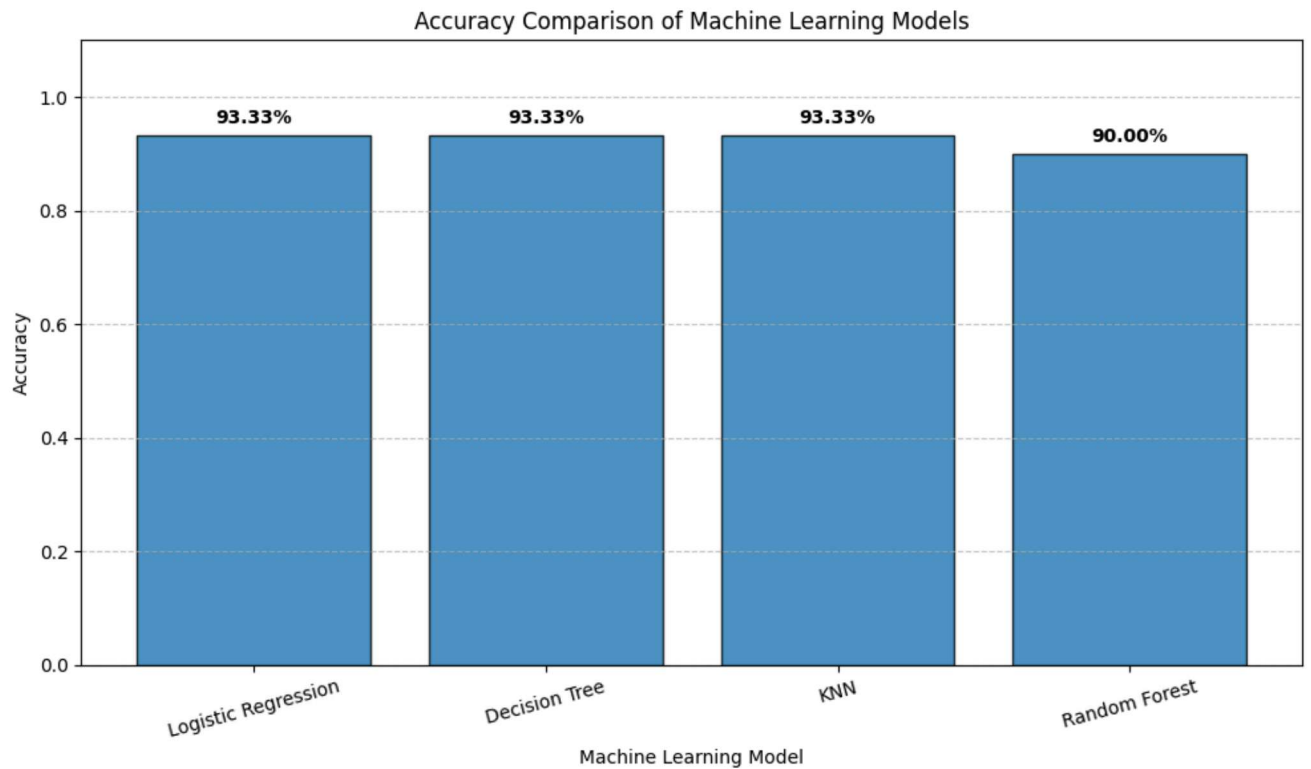
for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width()/2,
        height + 0.02,
        f"{height:.2%}",
        ha='center',
        fontweight='bold'
    )

plt.xticks(rotation=15)
plt.tight_layout()

plt.savefig("accuracy_comparison.png", dpi=300)

plt.show()

```



```
x = np.arange(len(results_df["Model"]))
width = 0.25

plt.figure(figsize=(11, 6))

plt.bar(
    x - width,
    results_df["Precision"],
    width,
    label="Precision",
    edgecolor='black'
)

plt.bar(
    x,
    results_df["Recall"],
    width,
    label="Recall",
    edgecolor='black'
)

plt.bar(
    x + width,
    results_df["F1 Score"],
    width,
    label="F1 Score",
    edgecolor='black'
)

plt.xlabel("Machine Learning Model")
plt.ylabel("Score")
plt.title("Precision, Recall and F1 Score Comparison")

plt.xticks(x, results_df["Model"], rotation=15)

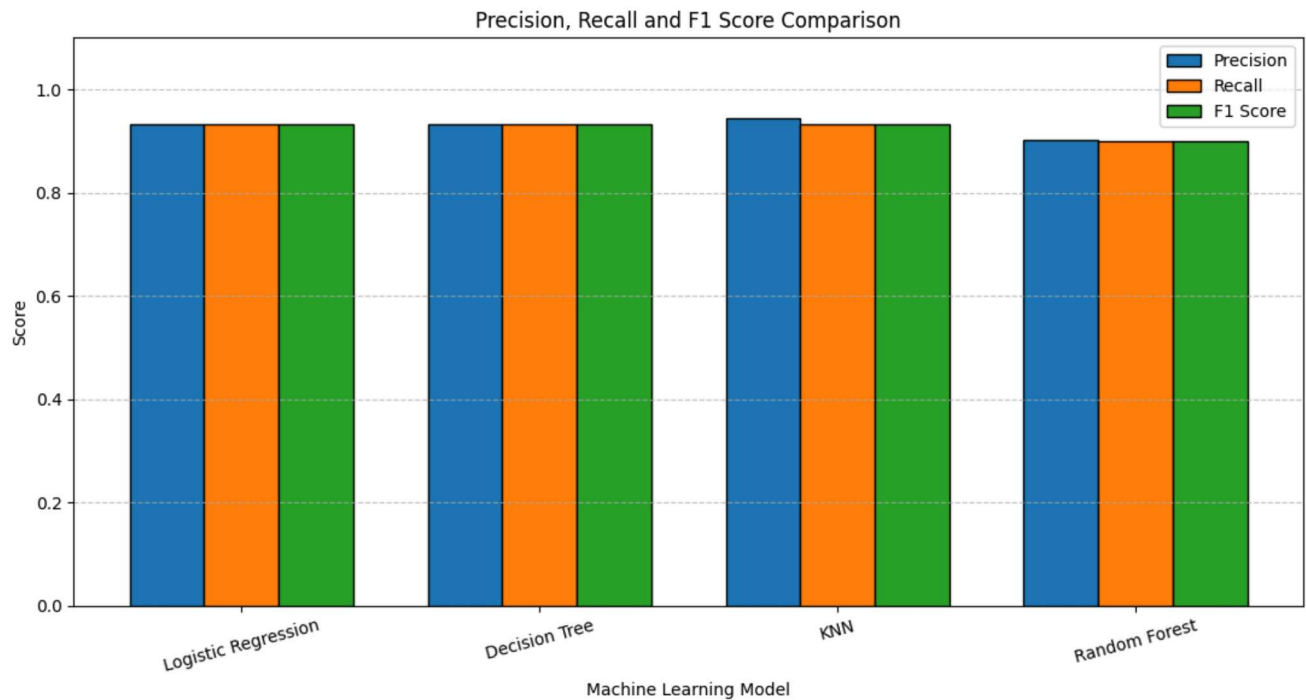
plt.ylim(0, 1.1)

plt.grid(
    axis='y',
    linestyle='--',
    alpha=0.7
)

plt.legend()
```

```
plt.tight_layout()
```

```
plt.show()
```



```
plt.figure(figsize=(10, 6))
```

```
plt.plot(
    results_df["Model"],
    results_df["Accuracy"],
    marker='o',
    linestyle='-',
    linewidth=2,
    markersize=8,
    label="Accuracy"
)
```

```
plt.xlabel("Machine Learning Model")
plt.ylabel("Accuracy")
plt.title("Machine Learning Model Accuracy Trend")
```

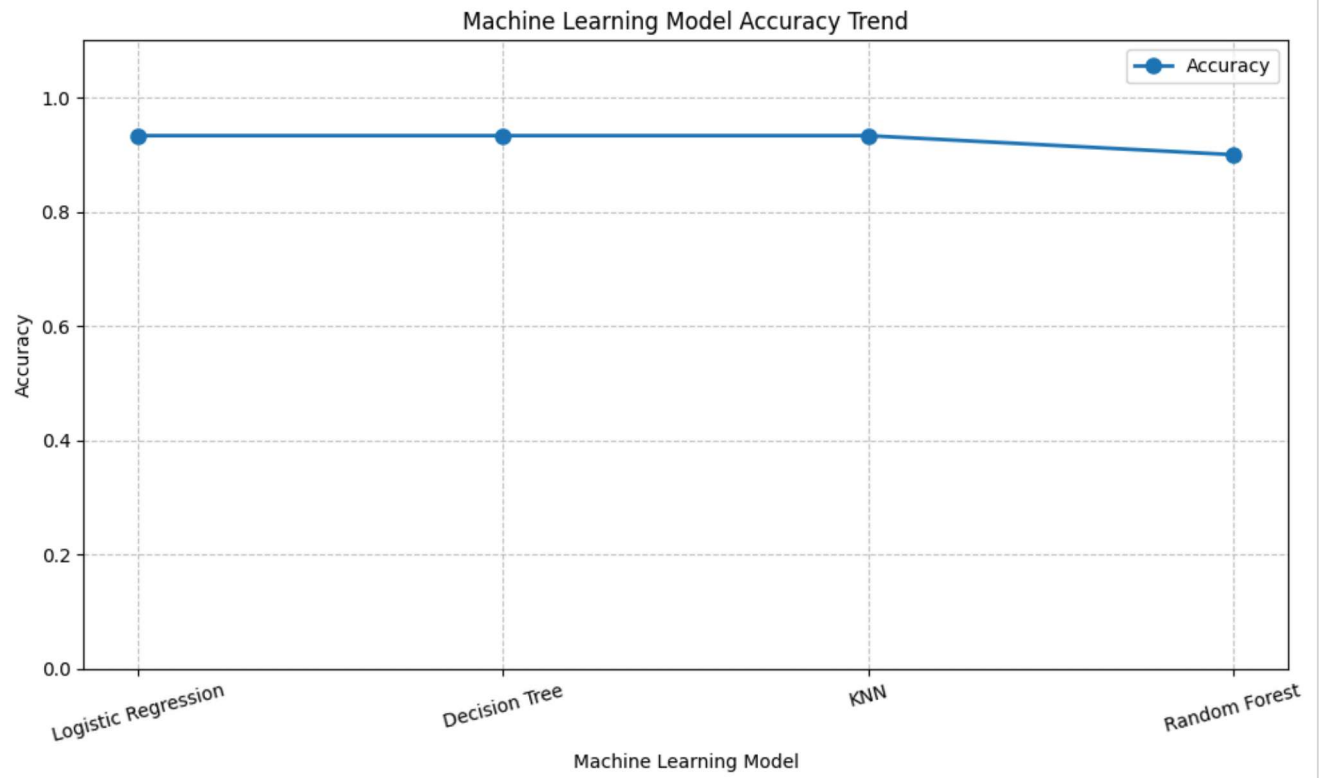
```
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend()
```

```
plt.ylim(0, 1.1)
```

```
plt.xticks(rotation=15)
```

```
plt.tight_layout()
```

```
plt.show()
```



```
best_model = results_df.loc[  
    results_df["Accuracy"] == max(results_df["Accuracy"])
```